

Day 10 — Mon 14 Sep

Countdown: 12 days to General English & GK

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

🕒 GK — Article 371A & Nagaland

Source: General/03_GK_CAPSULE.md

1.11 Panchayati Raj & local government — with the Nagaland twist

- **73rd Amendment (1992)** — rural local government (Panchayati Raj), **Part IX, Art. 243–243O**, 11th Schedule (29 subjects). Three tiers: Gram Panchayat → Panchayat Samiti (block) → Zilla Parishad (district).
- **74th Amendment (1992)** — urban local bodies (Municipalities), Part IXA, 12th Schedule (18 subjects).
- ★ **Nagaland link (exam gold):** By **Article 243M, Part IX (Panchayati Raj)** does **NOT apply to Nagaland, Meghalaya and Mizoram** and to the Scheduled/Tribal areas. Nagaland instead runs governance through its own **traditional Village Councils and Village Development Boards (VDBs)**, and its distinctive **Communitisation of Public Institutions and Services Act, 2002** — a genuinely Nagaland-specific innovation that hands local communities charge of schools, health centres and utilities. **This connects Section 1 to Section 2/3 and is a natural short-answer question.**

2.1 The intuition — why 371A exists at all

When Nagaland became a state in 1963, the Nagas had just come out of a long conflict with the Indian state over identity and autonomy. Article 371A was the constitutional promise that **statehood would not mean the erasure of Naga customary law, land ownership and social practice**. In plain terms: *Parliament's ordinary laws will not automatically override Naga tradition — the Nagaland Assembly gets to decide whether they apply*. It is a shield for Naga customary autonomy inside the Indian Union.

3.5 Geography of Nagaland

Feature	Detail
Terrain	Hilly, part of the Naga Hills / Patkai ranges; young fold mountains
Highest peak	Mount Saramati (~3,841 m) △ verify altitude , on the Myanmar border . Kiphire district
Dzükou Valley	Famous valley on the Nagaland–Manipur border near Kohima; the Dzükou lily ; trekking
Japfü peak	Second-highest, near Kohima; famous rhododendron trees
Major rivers	Doyang (longest river <i>in</i> Nagaland △ verify), Dhansiri , Tizu , Zungki , Milak
River drainage	Doyang & Dhansiri drain westward to the Brahmaputra system; Tizu drains eastward to the Chindwin (Myanmar) — a good discriminator fact
Intanki National Park	In Peren district — the state's only national park △ verify sole-status

Fakim Wildlife Sanctuary	Kiphire district
Other protected areas	Puliebadze WLS, Rangapahar WLS △ verify
Climate	Monsoon; heavy rainfall; pleasant summers in the hills

6.5 Nagaland geography

Cross-linked to §3.5 — Saramati, Dzükou, Doyang/Dhansiri/Tizu, Intanki NP, the Naga Hills, jhum. Know these both as "Nagaland GK" and as "geography."

Geography — likely exam questions

- [5] Name the major soil types of India with one crop each.
- [5] What is the South-West Monsoon and why is it important?
- [10] Describe the major physical divisions of India.

Geography — MCQ traps (with answers)

Answers

1. Longest peninsular river? → **Godavari** 2. Two west-flowing peninsular rivers? → **Narmada and Tapi**
3. Highest peak in India? → **Kanchenjunga** (Sikkim) 4. Most widespread soil in India? → **Alluvial** 5.
Asia's oldest oil refinery? → **Digboi (Assam)** 6. Which monsoon brings most of India's rain? → **South-West Monsoon**

SECTION 7 — GENERAL SCIENCE

Intuition first for the whole section. State-GK science is class-8-to-10 level: definitions, units, one formula, and everyday applications. Do not over-study; nail the standard facts.

● Core — OS intro & processes

Source: `_Shared/CORE_03_OperatingSystems.md`

A3. Evolution, generations and types of operating system

Concept

The history of operating systems is really the history of **one question: how do we stop the very expensive CPU from sitting idle?** Every generation is an answer to that question, and each answer creates the problem that the next generation solves. Learn the *sequence of motivations* and you never have to memorise the list.

```
No OS (plugboards)      → CPU idle while humans mount tapes
  ↓ automate job setup
Batch systems           → CPU idle during a job's own I/O
  ↓ keep several jobs in memory
Multiprogramming        → users must wait hours for a result; no interaction
  ↓ give each user a short slice of CPU
Time-sharing             → one CPU is still a bottleneck
  ↓ add more CPUs
```

Multiprocessing → machines are separate and networked
 ↓ hide the network
 Distributed / network OS → some jobs must meet hard deadlines
 ↓ guarantee response time
 Real-time OS → machines are small, battery powered
 ↓
 Embedded / mobile OS

The generations

Generation	Years	Technology	OS characteristics
First	1945–55	Vacuum tubes, plugboards	No operating system at all. Programmer = operator. Machine language only.
Second	1955–65	Transistors, punched cards, tape	Batch systems; resident monitor; JCL; FMS, IBSYS
Third	1965–80	ICs, multiprogramming	Multiprogramming, spooling, time-sharing; OS/360, MULTICS, UNIX
Fourth	1980–present	LSI/VLSI, PCs	Personal computer OS, GUI, networking; MS-DOS, Windows, Linux, macOS
Fifth (often listed)	present	Mobile/cloud/AI	Mobile, distributed, cloud, virtualisation; Android, iOS, hypervisors

The types — the table the exam actually wants

Type	Core idea	How the CPU is kept busy	Advantages	Disadvantages	Examples
Serial / simple batch	Jobs with similar needs are batched on tape and run one after another by a resident monitor; no interaction with the user once submitted	Poorly — CPU idles during that job's I/O	Simple; no idle operator time between jobs	Long turnaround; no interaction; a job can monopolise the machine; CPU still idle during I/O	Early IBM 1401/7094 setups
Multiprogrammed batch	Several jobs are kept in main memory at once; when the running job blocks for I/O, the OS switches the CPU to another job	Very well — CPU and I/O overlap	High CPU utilisation and throughput	Needs memory management, CPU scheduling and protection; complex	OS/360 MVT
Multitasking / time-sharing	Multiprogramming plus rapid switching so each of many interactive users gets the illusion of a private machine	Same as multiprogramming, but switching is driven by a time quantum , not only by I/O	Interactive; short response time; resource sharing among users	Overhead of frequent context switches; needs swapping/virtual memory; security between users is essential	UNIX, CTSS, MULTICS, Linux, Windows

Multiprocessing (parallel/tightly coupled)	Two or more CPUs share a common memory, bus and clock , under one OS	More CPUs = genuine parallel execution	Increased throughput ; economy of scale (share peripherals and memory); graceful degradation / fault tolerance	Complex; more OS overhead; speed-up is sub-linear (Amdahl's law)	SMP servers, multi-core PCs
Real-time (RTOS)	Correctness depends on meeting a deadline , not just producing the right answer	Priority-driven pre-emption; predictable, bounded latency	Guaranteed response time ; deterministic	Little or no virtual memory/ secondary storage; limited multitasking; expensive to design	VxWorks, RTLinux, QNX; used in avionics, missile control, medical, robotics, industrial control
Distributed (loosely coupled)	Several autonomous machines, each with its own memory , connected by a network, presented to the user as one system	Jobs are placed on whichever node is free	Resource sharing; speed-up; reliability (one node down ≠ system down); incremental growth	Network dependence; hard to design; security across nodes	Amoeba, LOCUS; modern cluster/ cloud platforms
Network OS	Machines stay visibly separate ; the OS just adds networking services (file/print sharing, remote login)	—	Simple, stable, easy to upgrade servers	User is aware of the multiplicity of machines; must know where resources live	Novell NetWare, Windows Server
Clustered	Multiple whole computers coupled to work together, usually sharing storage	—	High availability (failover), high performance	Complex cluster management software	Beowulf, Windows Failover Clustering
Embedded / mobile	OS for a special-purpose device with tight memory, power and cost limits	—	Small footprint, low power, real-time capable	Limited generality; often no user-installable software	Android, iOS, embedded Linux, RTOS in appliances

The three "multi-" words — the single most common confusion

Term	What is multiple	One-line definition
Multiprogramming	Programs in memory	More than one job resides in main memory; when one blocks for I/O the CPU is switched to another. Goal: maximise CPU utilisation.

Multitasking (time-sharing)	Tasks getting CPU turns rapidly	Logical extension of multiprogramming where the CPU switches on a time quantum so users get interactive response. Goal: minimise response time.
Multiprocessing	CPUs (processors)	Two or more physical processors under one OS sharing memory. Goal: true parallelism and throughput.
<i>(Multithreading)</i>	<i>Threads within one process</i>	<i>One process split into several independently schedulable threads sharing its address space.</i>

Say it like this in the exam: multiprogramming multiplies *programs*, multitasking multiplies *turns*, multiprocessing multiplies *processors*, multithreading multiplies *threads inside one process*. That sentence alone has earned candidates full marks on a 5-mark question.

Asymmetric vs symmetric multiprocessing

	Asymmetric (ASMP)	Symmetric (SMP)
Role of CPUs	One master CPU runs the OS and assigns work; others are slaves	All CPUs are peers ; each runs a copy of the OS and can perform any task
Failure of a CPU	Master failure is fatal	Graceful degradation
Complexity	Simpler	More complex (needs careful locking)
Used in	Older/large systems, some embedded	Virtually all modern multi-core systems

Hard vs soft real-time

	Hard real-time	Soft real-time
Deadline	Must be met; a miss is a system failure	Should be met; a miss degrades quality only
Secondary storage / virtual memory	Usually absent (unpredictable delays)	Allowed
Examples	Airbag deployment, nuclear reactor control, flight control, pacemaker	Video streaming, VoIP, multimedia playback, online gaming

Likely exam questions

- **[5]** Distinguish between multiprogramming, multitasking and multiprocessing.
- **[5]** What is a real-time operating system? Differentiate hard and soft real-time systems.
- **[5]** Explain batch processing systems and their limitations.
- **[10]** Trace the evolution of operating systems from serial processing to distributed systems, explaining what problem each stage solved.
- **[10]** Explain the different types of operating systems with their advantages, disadvantages and examples.
- **[15]** Describe multiprogramming, time-sharing, multiprocessing, real-time and distributed operating systems in detail, comparing them on CPU utilisation, response time, hardware requirement and application area.

MCQ traps

Trap	Correct answer
"Multiprogramming needs multiple CPUs"	No — one CPU, several programs in memory.
"Time-sharing = multiprogramming"	Time-sharing is multiprogramming plus time-quantum switching for interactivity.
"Main objective of multiprogramming"	Maximise CPU utilisation (not response time — that is time-sharing).
"Main objective of time-sharing"	Minimise response time.
"RTOS is one that runs very fast"	It is one that responds within a guaranteed deadline — predictability, not raw speed.
"In a distributed system, processors share a common clock and memory"	No — that is <i>multiprocessing</i> . Distributed systems are loosely coupled ; no shared memory or clock.
"Spooling is a form of buffering that requires..."	Spooling uses the disk as a huge buffer; buffering usually uses memory .
"First generation OS"	There was no OS in the first generation.
"Which OS type has graceful degradation?"	Multiprocessing (and clustered).

PART B — PROCESS MANAGEMENT

B1. The process concept

Concept — plain English first

A **program** is a file on disk: a passive lump of instructions. Nothing about it changes over time. The moment you run it, the OS creates a *living instance* of it — code loaded into memory, a program counter pointing at the current instruction, a stack of function calls, some heap memory, a set of open files, a user identity, a priority. **That living instance is a process.** The same program run three times gives you three processes: same code, three completely separate sets of everything else.

The classic analogy: the program is the **recipe**; the process is the **act of cooking** it. One recipe, three cooks, three simultaneous cooking processes, each at a different step with its own bowls and ingredients.

Formal definition

A **process** is a program in execution — an active entity comprising the program code (text section), the current activity represented by the value of the program counter and processor registers, a stack (temporary data: parameters, return addresses, local variables), a data section (global variables) and a heap (dynamically allocated memory).

Program vs process — the table

	Program	Process
Nature	Passive entity	Active entity
Location	Stored on secondary storage (disk)	Loaded in main memory

CPU-scheduling information	Priority, pointers to scheduling queues, scheduling parameters
Memory-management information	Base and limit registers, page tables or segment tables
Accounting information	CPU time used, real time elapsed, time limits, account numbers, job/process numbers
I/O status information	List of open files (file descriptor table), allocated I/O devices, pending I/O requests
Pointers	To parent, children, and to the next PCB in a queue
Privileges / ownership	UID, GID, security context

THE PCB — draw this box in the exam

Process ID / Parent PID
Process state
Program counter
CPU registers
CPU-scheduling information
Memory-management info
Accounting information
I/O status information
Pointer to next PCB

The set of all PCBs is the **process table**.

B2. Process states and transitions

Concept

A process does not run continuously from start to finish. It runs a bit, asks for data from disk, waits, gets the data, waits for the CPU to become free again, runs a bit more. The OS tracks where in that cycle each process is by giving it a **state**.

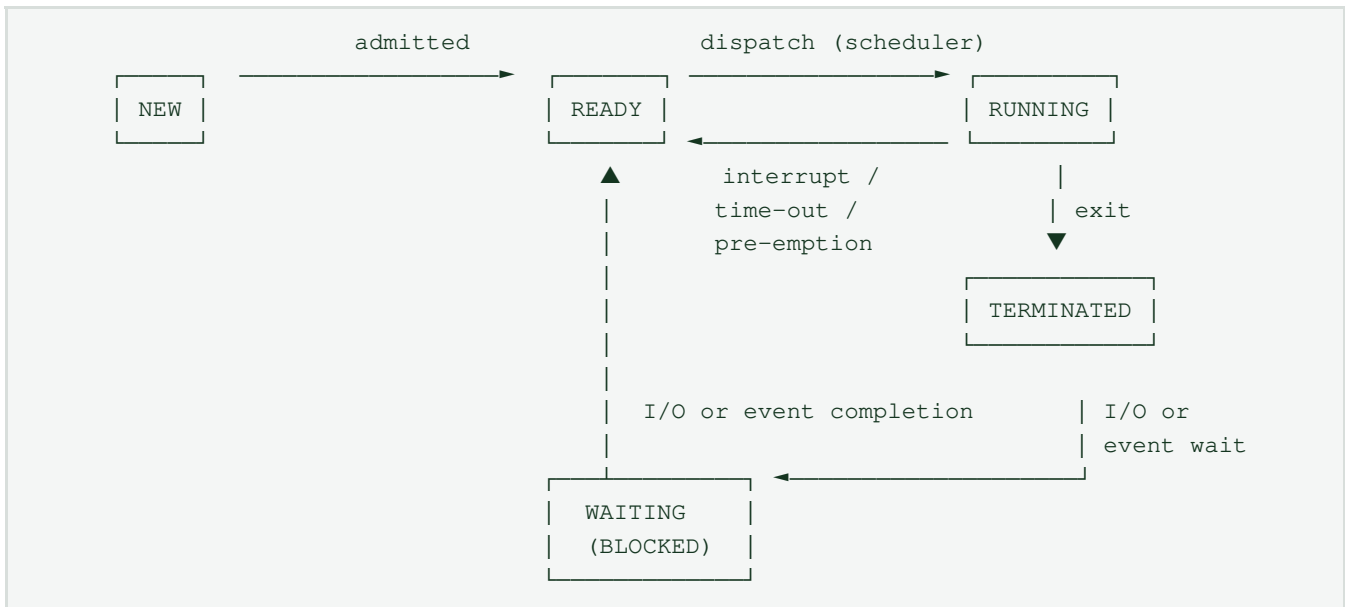
The crucial insight for the exam: **there are exactly two things that can take a process out of the Running state, and they lead to two different places**. If it is *forced off* by the scheduler (its quantum expired, or a higher-priority process arrived) it goes to **Ready** — it could still run, it just isn't allowed to. If it *voluntarily* gives up the CPU to wait for something (I/O, a signal, a resource) it goes to **Waiting/Blocked** — it *cannot* run even if the CPU were free. That distinction is worth a mark every time.

The five-state model

State	Meaning
New	The process is being created; its PCB exists but it has not yet been admitted to the ready queue

Ready	Runnable and waiting only to be assigned a processor
Running	Instructions are being executed (on a single-CPU system, exactly one process is Running)
Waiting / Blocked	Waiting for some event — I/O completion, a semaphore, a child to terminate, a signal
Terminated / Exit	Execution has finished; the OS is reclaiming resources

The state diagram — hand-draw this exactly



Label every arrow. The five transitions and their causes:

#	Transition	Cause
1	New → Ready	Admit — the long-term scheduler accepts the process
2	Ready → Running	Dispatch — the short-term scheduler (dispatcher) selects it
3	Running → Ready	Time-out / pre-emption / interrupt — quantum expired or a higher-priority process arrived. <i>Involuntary.</i>
4	Running → Waiting	Event wait — issues an I/O request or a <code>wait()</code> . <i>Voluntary.</i>
5	Waiting → Ready	Event completion — the I/O finished or the awaited event occurred
6	Running → Terminated	Exit — normal completion or abort

Impossible transitions — a favourite MCQ.

- **Ready → Waiting** is impossible: a Ready process is not executing, so it cannot issue an I/O request.
- **Waiting → Running** is impossible: after its event completes, a process must go through **Ready** and be selected by the scheduler.

The seven-state model (with swapping)

When main memory is short, the medium-term scheduler **swaps** whole processes out to disk. This adds two suspended states:

Extra state	Meaning

Ready-suspended	Runnable but currently swapped out to secondary storage
Blocked-suspended	Waiting for an event and swapped out

Transitions: Ready ↔ Ready-suspended (**suspend / resume**), Blocked ↔ Blocked-suspended, and crucially **Blocked-suspended** → **Ready-suspended** when the awaited event completes while the process is still on disk.

Additional UNIX-specific states worth one line

- **Zombie**: the process has terminated but its parent has not yet called `wait()`, so its exit status is still held in the process table. It consumes a PID but no other resources.
- **Orphan**: the parent terminated first; the process is **re-parented to** `init/systemd` (PID 1), which reaps it.

Likely exam questions

- [5] Differentiate between a program and a process.
- [5] What is a PCB? List the information stored in it.
- [5] Why can a process not move directly from Waiting to Running?
- [10] Draw and explain the process state transition diagram, describing each state and every transition with its cause.
- [10] Explain the seven-state process model with suspended states and swapping.
- [15] Explain the process concept in detail: process vs program, the memory layout of a process, the process control block with all its fields, the five-state diagram with all transitions, and the role of the three schedulers.

MCQ traps

Trap	Correct answer
"How many processes can be in Running state on a single CPU?"	Exactly one.
"Ready → Blocked is a valid transition"	Invalid.
"Blocked → Running is valid"	Invalid — must pass through Ready.
"The PCB is stored in the process's own stack"	It is stored in kernel memory (the process table).
"A zombie process consumes CPU"	It consumes only a process-table entry .
"Which is NOT in the PCB?"	The program's source code . (The <i>text section</i> is in the process's memory, not in the PCB.)
"An orphan process is killed"	It is adopted by init (PID 1) .

B4. Threads

Concept

A process is expensive: its own address space, its own page tables, its own file table.

But often you want several activities inside **one** application that share all of that —

a word processor that formats text, checks spelling and autosaves at the same time, all

working on the *same document in the same memory*. Creating three processes would mean three

copies of the document and painful IPC to keep them in sync.

A thread is a lightweight process: an independent path of execution (its own program counter, register set and stack) that shares the process's code, data, heap and open files with its sibling threads. One process, many threads, one address space.

What is shared and what is private

Shared by all threads of a process	Private to each thread
Code / text section	Program counter
Data section (globals)	Register set
Heap	Stack
Open files / file descriptors	Thread ID
Signals and signal handlers	Priority / scheduling state
Current working directory, UID/GID	Errno, thread-local storage

Process vs thread

	Process	Thread
Also called	Heavyweight process	Lightweight process (LWP)
Address space	Own, isolated	Shared with siblings
Creation cost	High	Low (order of 10–100× cheaper)
Context switch cost	High (page tables, TLB flush)	Low (no memory-map change)
Communication	Needs IPC (pipes, shared memory, messages) — kernel involvement	Direct via shared variables — but needs synchronisation
Protection	Processes are protected from each other	No protection between threads — one bad thread can corrupt the whole process
Effect of a crash	Other processes unaffected	Can bring down the entire process
Blocking	One process blocking does not block others	With user-level threads , one blocking call blocks all threads of the process

Benefits of multithreading — four standard points

1. **Responsiveness** — the UI thread stays alive while another thread does the long job.
2. **Resource sharing** — threads share memory and files by default; no IPC set-up needed.
3. **Economy** — far cheaper to create and context-switch than processes.
4. **Scalability / utilisation of multiprocessor architectures** — threads of one process can genuinely run in parallel on different cores. (A single-threaded process can only ever use one core.)

User-level vs kernel-level threads

	User-level threads (ULT)	Kernel-level threads (KLT)

Managed by	A thread library in user space; kernel is unaware	The kernel itself
Switching	Fast — no mode switch to kernel	Slower — requires kernel intervention
Blocking system call	Blocks the entire process (all its threads)	Blocks only that thread
True parallelism on multiple CPUs	No (kernel sees one schedulable entity)	Yes
Scheduling	Application-specific scheduling possible	Kernel's scheduler
Portability	High (library-level)	OS-dependent
Examples	POSIX Pthreads (can be either), Java green threads, older Solaris	Windows threads, Linux tasks, modern Solaris/Linux Pthreads

Multithreading models

Model	Mapping	Notes	Example
Many-to-One	Many ULTs → one KLT	Efficient switching but no parallelism ; one blocking call blocks everything	Green threads
One-to-One	Each ULT → its own KLT	True concurrency ; but creating a user thread creates a kernel thread → overhead limits thread count	Linux, Windows
Many-to-Many	m ULTs → n KLTs ($n \leq m$)	Best of both: parallelism <i>and</i> unlimited user threads; complex to implement	Older Solaris, Windows with ThreadFiber
Two-level	Many-to-many plus the ability to bind a specific user thread to a kernel thread	Hybrid	IRIX, HP-UX

Likely exam questions

- **[5]** Define a thread. What resources does it share with other threads of the same process?
- **[5]** State any four benefits of multithreading.
- **[5]** Differentiate between user-level and kernel-level threads.
- **[10]** Distinguish between a process and a thread. Explain the multithreading models with diagrams.
- **[15]** What is a thread? Explain why threads are called lightweight processes. Describe user-level and kernel-level threads, the four multithreading models, and the benefits and problems of multithreaded programming.

MCQ traps

Trap	Correct answer
"Threads share the stack"	No — each thread has its own stack . They share code, data, heap and files.
"Threads have their own address space"	No — that is a process.
"A thread is a heavyweight process"	Lightweight.

"Many-to-one gives true parallelism"	No — only one-to-one and many-to-many do.
"Linux uses many-to-many"	One-to-one.
"Threads of a process are protected from each other"	They are not.